

Kapitel 1: Kom igång med Python

Viktiga Begrepp och Studieförfrågor (DA2005)

Studieguide

July 23, 2026

1 Kapitel 1: Intro till Python

Programmering

Att skriva instruktioner (program) som en dator sedan kan köra för att lösa specifika problem.

Anaconda

En distribution och ett verktygsprogram för Python som används för att hantera paket, bibliotek och virtuella utvecklingen.

Spyder

En integrerad utvecklingsmiljö (IDE) som följer med Anaconda, utrustad med kodredigering, syntaxmarkering (syntax highlighting) och en interaktiv konsol.

Python-tolk (Interpreter)

Det program som läser, analyserar och kör (tolkar) Python-kod rad för rad i realtid.

Konsol (Console)

Det interaktiva fönstret (t.ex. i Spyder) där man kan skriva enstaka Python-uttryck för att direkt se deras resultat, samt där programmets utskrifter visas.

Variabel

En namngiven behållare i datorns minne som används för att spara och referera till ett värde.

Tilldelning (Assignment)

Processen där ett värde lagras i en variabel med hjälp av tilldelningsoperatoren `=`. Exempel:
`s = 67`.

Datatyp

En klassificering av data som talar om för tolken hur värdet ska användas. De tre primära typerna i kapitel 1 är:

- **Heltal (int):** Representerar heltal utan decimaler (t.ex. 17, -3).
- **Flyttal (float):** Representerar reella tal med decimalpunkt (t.ex. 3.14, 1.25).
- **Sträng (str):** En sekvens av tecken som omges av enkelfnuttar ('`hej`') eller dubbelfnuttar ("`hej`").

Aritmetiska operationer

Matematiska operationer för beräkningar:

- **Heltalsdivision (//):** Dividerar två tal och avrundar nedåt till närmaste heltal.
- **Exponentiering (**):** Upphöjt till (t.ex. `2**3` ger 8).

- **Modulus/Rest (%)**: Returnerar resten vid en heltalsdivision (t.ex. `5 % 2` ger 1).

Reserverade ord (Keywords)

Ord som har en specifik fördefinierad betydelse i Python och som därför inte får användas som variabelnamn (t.ex. `and`, `if`, `for`, `True`).

Identifierare (Identifiers)

Namn på resurser i programmet, såsom variabler, funktioner eller klasser. De måste börja med en bokstav eller understreck (`_`) och får inte börja med en siffra.

Litteral

Ett fast, konstant värde som skrivs direkt i programkoden (t.ex. talet `2.5` eller strängen `'SU'`). Bör helst inte spridas okontrollerat i koden (undvik s.k. "hårdkodning").

Instuderingsfrågor (Review Questions)

Fråga 1: Vad är skillnaden mellan en Python-tolk och en utvecklingsmiljö som Spyder?

Svar: Tolken kör själva koden, medan Spyder är ett verktyg/redigeringsprogram som gör det lättare för programmeraren att skriva, felsöka och organisera koden.

Fråga 2: Varför är ordningsföljden på instruktioner viktig vid tilldelning av variabler? Ge ett exempel på en otillåten ordning.

Svar: Python körs sekventiellt (rad för rad). Du kan inte använda en variabel i en beräkning innan den har definierats. Till exempel ger följande kod felmeddelandet `NameError`:

```
v = s / t # Fel! s och t är inte definierade än.
s = 67
t = 1.25
```

Fråga 3: Förklara skillnaden i resultat mellan operatorerna `/`, `//`, och `%` med hjälp av talen 7 och 3.

Svar:

- `7 / 3` ger flyttalet 2.3333333333333335 (vanlig division).
- `7 // 3` ger heltalet 2 (heltalsdivision, kapar decimalerna).
- `7 % 3` ger resten 1 (modulus, eftersom $7 = 2 \times 3 + 1$).

Fråga 4: Vad händer om du kör koden `'hej' * 3` respektive `'hej' + 3`? Förklara varför.

Svar:

- `'hej' * 3` duplicerar strängen och ger `'hejhejhej'`. Detta är en tillåten operation (sträng multiplicerat med heltal).
- `'hej' + 3` ger ett `TypeError` eftersom man inte kan addera (konkatenera) en sträng med ett heltal (`int`) direkt utan att först konvertera talet till en sträng.

Fråga 5: Vilka av följande är giltiga identifierare (variabelnamn) i Python? Om de är ogiltiga, förklara varför.

a) `my_variable`

- b) `2nd_value`
- c) `class`
- d) `Value_17`

Svar:

- a) och d) är **giltiga**.
- b) är **ogiltig** eftersom en identifierare inte får börja med en siffra.
- c) är **ogiltig** eftersom `class` är ett reserverat ord (keyword) i Python.

Fråga 6: Vad menas med att “hårdkoda” ett värde, och varför bör man undvika litteraler spridda i koden enligt kompendiet?

Svar: Att hårdkoda innebär att skriva in konstanta värden (litteraler) direkt i beräkningar spridda över programmet. Det bör undvikas eftersom det försvårar uppdateringar (om värdet ändras måste man leta upp alla ställen manuellt) och ökar risken för programmeringsfel. Det är bättre att definiera värdet en gång i en tydligt namngiven variabel längst upp i koden.

2 Kapitel 2: intro till python

Syntax

Regler för hur kod skrivs för att Python ska förstå den.

Semantik

Innebörden eller betydelsen av vad koden gör.

Datatyper

Python har inbyggda typer som `str`, `int`, `float`, `complex`, `bool` och `NoneType`.

Typomvandling

Kan vara implicit (automatisk) eller explicit (t.ex. `int(x)`).

Kortslutning (Short circuiting)

När logiska operatorer (`and/or`) avbryter utvärderingen så fort resultatet är garanterat.

Modulär kod

Program uppdelade i mindre, fristående funktioner.

Docstrings

Dokumentationssträngar knutna till funktioner, läsbara via `help()`.

Scope

Räckvidden för en variabel (global eller lokal).

Fråga 1: Vad är skillnaden mellan `SyntaxError` och semantiska fel?

Svar: Syntaxfel innebär att koden inte följer språkets grammatik, vilket gör att den inte kan köras. Semantiska fel innebär att koden är korrekt skriven men utför fel logik, vilket resulterar i oväntade utdata.

Fråga 2: Varför representeras flyttal som approximationer?

Svar: Python representerar flyttal genom bråk i bas 2. Det är matematiskt omöjligt att representera alla reella tal exakt med ett begränsat antal bitar på en dator.

Fråga 3: Hur ändrar man en global variabel inuti en funktion?

Svar: Genom att använda det reserverade ordet **global** inuti funktionen deklarerar man att man avser att ändra den globala variabeln snarare än att skapa en lokal variabel med samma namn.

Fråga 4: Vad händer om en funktion saknar return?

Svar: Om **return** saknas returnerar funktionen värdet **None** som standard.

Fråga 5: Varför är `elif` användbart?

Svar: Det förbättrar kodens läsbarhet genom att ersätta krångliga, nästlade **if-else**-strukturer, vilket gör logiken tydligare att följa.

3 Kapitel 3: Listor och iteration

Listor

En sekvens av element (av valfri typ) inneslutna i hakparenteser `[]`.

Iteration

Processen att automatisera repetitiva uppgifter.

for-loop

Används för att iterera över element i en sekvens.

while-loop

Används för att repetera kod så länge ett villkor är sant.

Fråga 1: Varför är listor viktiga?

Svar: De gör det möjligt att samla och hantera flera element effektivt, vilket underlättar beräkningar och databehandling.

Fråga 2: Vad är skillnaden mellan `for` och `while`?

Svar: **for**-loopar används för att iterera över samlingar, medan **while**-loopar repeterar kod baserat på ett sant villkor.

4 Kapitel 4: Filer och dictionaries

Datastruktur

En struktur för att organisera och samla data logiskt.

Uppslagstabell

En datastruktur som kan indexeras med andra typer än bara heltal.

Filhantering

Tekniker för att läsa från och skriva till filer på disk.

Besvarade Instuderingsfrågor

Fråga 1: Vad skiljer en uppslagstabell från en lista?

Svar: Listor indexeras via heltal, medan uppslagstabeller tillåter indexering via andra datatyper.

Fråga 2: Vad är syftet med filhantering?

Svar: Det möjliggör permanent lagring av data samt inläsning av extern data som inte finns direkt i programkoden.

5 Kapitel 5: Felhantering och undantag

Felhantering

Metoder för att hantera programfel (t.ex. krascher).

Särfall (Exception)

Signalering av fel som uppstått under körning.

try/except

Pythons verktyg för att fånga och hantera uppkomna fel.

Listomfattning

Kompakt syntax för att skapa och bearbeta listor.

Fråga 1: Varför är särfall bättre än felkoder?

Svar: Felkoder gör koden svåräst och komplex p.g.a. ständiga kontroller. Särfall separerar felhantering från den ordinarie logiken.

Fråga 2: Vad är fördelen med listomfattning?

Svar: Det möjliggör mer kompakt och effektiv kod vid skapande av listor jämfört med traditionella loopar.

6 Kapitel 6: Sekvenser och generatorer

Sekvenstyp

Datatyp med gemensamma operationer (t.ex. indexering, slice).

Muterbar

Objekt som kan ändras efter skapande (t.ex. listor).

Icke-muterbar

Objekt som är konstanta (t.ex. tupler, strängar).

Fråga 1: Vad är skillnaden mellan muterbara och icke-muterbara objekt?

Svar: Muterbara objekt kan ändras efter skapande, medan icke-muterbara objekt är konstanta.

Fråga 2: Varför är muterbarhet viktigt?

Svar: Det påverkar hur objekt hanteras i funktioner och kan leda till oväntade sideffekter om man inte är medveten om att originalobjektet ändras.

7 Kapitel 7: Moduler, bibliotek och programstruktur

Sekvenstyp

Datatyp med gemensamma operationer (t.ex. indexering, slice).

Muterbar

Objekt som kan ändras efter skapande (t.ex. listor).

Icke-muterbar

Objekt som är konstanta (t.ex. tupler, strängar).

Fråga 1: Vad är skillnaden mellan muterbara och icke-muterbara objekt?

Svar: Muterbara objekt kan ändras efter skapande, medan icke-muterbara objekt är konstanta.

Fråga 2: Varför är muterbarhet viktigt?

Svar: Det påverkar hur objekt hanteras i funktioner och kan leda till oväntade sideffekter om man inte är medveten om att originalobjektet ändras.

```
1      '''
2      short/modular (variables, functions, files)
3      variables (nouns, repeating stuff should be global constants)
4      functions (action on what, return for local > global=constant, keep
        be math alone, don't return int/bool/fixed stuff/unspecified
        numbers, several returns > one)
5      documentation (what, arg & returns - name, type, description)
6      OOP (code skeleton)
7      Tests
8      ! main
9      commenting (why you chose a method, use this when) - no syntax
        explanations, no examples, no long sentences
10
11     Type hinting (def(arg: type) -> return_type)
12     '''
```

8 Kapitel 8: Funktionell programmering

Funktionell programmering

bygger på att man anropar, sätter ihop och modifierar funktioner.

Rekursion

En teknik där en funktion anropar sig själv för att lösa ett problem.

Högre ordningens funktioner

Funktioner som tar andra funktioner som argument eller returnerar funktioner.

Anonyma funktioner (lambda)

Funktioner utan namn som definieras med `lambda`-syntaxen. Används ofta som argument till högre ordningens funktioner då de inte behöver återanvändas.

Svansrekursion

En form av rekursion där det rekursiva anropet är det sista som sker i funktionen. Vissa språk kan optimera detta till en loop för att spara minne.

Fråga 1: Vad innebär rekursion?

Svar: Rekursion är när en funktion anropar sig själv. Det är ett alternativ till loopar och kan göra koden mer elegant.

Fråga 2: Vad kännetecknar funktionell programmering?

Svar: Det bygger på att använda, kombinera och modifiera funktioner snarare än att styra programflödet med loopar och variabler som ändras.

Fråga 3: Varför behöver rekursion ett basfall?

Svar: Ett basfall behövs för att stoppa rekursionen och förhindra oändliga anrop.

```
1     def pascal(n):
2         '''
3         Ex:
4         1. pascal(3) exekveras:
```

```

5     n = 3 (inte 1, gar vidare)
6     Anropar pascal(2) for att fa 'prev_row'
7 2. pascal(2) exekveras:
8     n = 2 (inte 1, gar vidare)
9     Anropar pascal(1) for att fa 'prev_row'
10 3. pascal(1) exekveras:
11     n = 1 (basfallet uppnatt!)
12     RETURVARDE FRAN pascal(1): [1]
13
14 [RETURFASEN - Vagen upp och berakning av varden]
15 4. Tillbaka i pascal(2):
16     Mottagit 'prev_row' = [1]
17     Initierar 'new_elements' = []
18     Beraknar loopen: range(len([1]) - 1) -> range(0) -> Loopen kors
19         0 ganger
20     Satter ihop raden: [1] + [] + [1]
21     RETURVARDE FRAN pascal(2): [1, 1]
22 5. Tillbaka i pascal(3):
23     Mottagit 'prev_row' = [1, 1]
24     Initierar 'new_elements' = []
25     Beraknar loopen: range(len([1, 1]) - 1) -> range(1) -> Loopen
26         kors for i = 0
27         * i = 0: prev_row[0] + prev_row[1] -> 1 + 1 = 2
28         * new_elements.append(2) -> 'new_elements' ar nu [2]
29     Satter ihop raden: [1] + [2] + [1]
30     RETURVARDE FRAN pascal(3): [1, 2, 1]
31
32 '''
33 if n == 1:
34     return [1]
35
36 prev_row = pascal(n - 1)
37 new_elements = []
38
39 for i in range(len(prev_row) - 1):
40     new_elements.append(prev_row[i] + prev_row[i+1])
41
42 return [1] + new_elements + [1]

```

9 Kapitel 9: OOP och klasser

Objektorientering

är ett programmeringsparadigm eller programmeringsstil där man samlar data och dataskrukturer med de funktioner som kan tillämpas på dem.

Klass, objekt, instans

En klass är en mall för att skapa objekt. Ett objekt är en instans av en klass (dvs. ett konkret exempel på klassen). Skillnaden mellan objekt och instans är att objekt är en mer generell term, medan instans syftar på ett specifikt objekt som skapats från en klass.

Attribut

En variabel som är knuten till ett specifikt objekt och beskriver dess nuvarande tillstånd. De är som objektets variabler. Det finns instansattribut (knutna till ett specifikt objekt) och klassattribut (delade av alla objekt i klassen).

Metod

En funktion som är knuten till ett specifikt objekt och kan manipulera dess attribut eller

utföra operationer relaterade till objektet. De är som funktioner till objektet.

Konstruktor

En speciell metod som används för att skapa och initiera objekt av en klass. I Python är konstruktorn metoden `__init__()`. Så när man anropar klassen igen för att skapa ett nytt objekt, körs konstruktorn automatiskt för att sätta upp objektets initiala tillstånd.

Metod

En funktion som är knuten till ett specifikt objekt.

Punktnotation

Syntaxen `objekt.metod()` för att anropa metoder.

Self

En referens till det aktuella klassens objektet, används för att komma åt objektets attribut och metoder. Kan kallas annat men `self` är konvention.

Privata och publika attribut/metoder

Publika attribut/metoder kan nås utanför klassen, medan privata är avsedda att endast användas inom klassen. I Python markeras privata med ett understreck (`_`) i början av namnet.

Modulär programmering

Att dela upp program i mindre, fristående enheter (moduler) som kan återanvändas och underhållas separat vilket leder till mindre kod att ändra vid behov.

```
1  # Syntax for classes
2  class KlassNamn: # Bör namnges med stor bokstav i början, va ett
   substantiv tex Bilar
3
4      competition = "Formula 1" # Klassattribut, delad av alla objekt
   i klassen
5
6      def __init__(self, arg1, arg2): # Instansattribut, unik för
   varje objekt genom att data läggs till
7          self.attribut1 = arg1 # Här skapas attribut/variabler tex å
   lder, vikt
8          self.attribut2 = arg2
9
10     def metod(self, attribut1, attribut2): # funktion/metod som ber
   äknar bilens värde
11         värde = self.attribut1 * 1000 + self.attribut2 * 500 # Obs
   även enkla funktioner som len() behövs läggas till så de
   funkar på din klass
12         return värde
13
14     # Lägg till enkla funktioner vi vill ha
15     def __str__(self): # __str__() är en speciell metod som
   definierar hur objektet ska representeras som en sträng
16         return f"Objekt med attribut1: {self.attribut1}, attribut2:
   {self.attribut2}, competition: {KlassNamn.competition}"
17
18     # Syntax för att skapa ett objekt (instans) av klassen
19     objekt = KlassNamn(ålder1, vikt2) # objekt kan nu manipulera hela
   datan från Bilar med hjälp av enkel punktnotation
20     objekt.attribut3 = värde3 # Här kan man lägga till fler attribut,
   obs kan skrivas över
```



```

21 objekt.metod() # Här kan man anropa metoder som redan finns i
    klassen
22 objekt2 = KlassNamn(ålder2, vikt3) # Här skapas ett nytt objekt med
    annan data som en kopia
23 print(objekt2.attribut2) # För att använda funktioner som redan
    finns på objektet, kalla på dess info/attribut

```

```

1 # Privata och publika attribut/metoder
2 class Bil:
3     def __init__(self, ålder, vikt):
4         self._ålder = ålder # Privata attribut (bör inte ändras
            utanför klassen)
5         self._vikt = vikt # Privata attribut (bör inte ändras
            utanför klassen)
6
7     def beräkna_värde(self):
8         return self._ålder * 1000 + self._vikt * 500 # Publik
            metod som använder privata attribut
9
10 min_bil = Bil(5, 1500) # man kan manipulera med beräkna_värde() men
    inte ändra _ålder eller _vikt direkt om man inte skriver...
11 min_bil._ålder = 10 # ...så här, men det är inte rekommenderat
    eftersom det bryter mot principen om inkapsling.

```

```

1 # Sets
2 a = {1, 2, 3, 4, 5} # Vilket är samma som {1, 1, 2, 3, 4, 5, 5}
3 b = {4, 5, 6, 7, 8}
4 print(a | b) # Union
5 print(a & b) # Intersection
6 print(a - b) # Difference
7 print(a ^ b) # Symmetric difference = (a - b) | (b - a)

```

Fråga 1: Varför använda objektorientering?

Svar: Knyta samman mycket data som relaterar till varandra (hör till samma objekt) vilket ger bra struktur med relaterad funktionalitet på samma ställe. Också att kunna definiera egna datatyper som hjälper abstraktion genom att kunna gömma detaljer.

10 Kapitel 10: OOP arv

Arv

Mekanism för att återanvända och utöka kod genom subklasser.

Basklass

Den ursprungliga klassen som funktionalitet ärvs ifrån.

Subklass

Den klass som ärver från en basklass.

Fråga 1: Varför använder vi arv?

Svar: För att återanvända kod på ett strukturerat sätt och för att kunna bygga vidare på existerande klasser.

Fråga 2: Hur definieras en subklass i Python?

Svar: Man anger basklassen inom parentes i subklassens definition, t.ex. `class Sub(Bas):`.

11 Kapitel 11: OOP arv mer

Multipelt arv

När en klass ärver från två eller fler superklasser.

Diamantproblemet

En tvetydighet i arvshierarkin som kan uppstå vid multipelt arv.

Fråga 1: Vad är diamantproblemet?

Svar: Det är en konflikt i arvshierarkin där en subklass ärver från två klasser som delar samma basklass, vilket skapar osäkerhet om vilken metod som ska anropas.

Fråga 2: Varför är multipelt arv kontroversiellt?

Svar: Eftersom det kan göra arvshierarkin oöverskådlig och skapa konflikter som inte finns i enkla arvsstrukturer.

12 Kapitel 12: Defensiv programmering

Defensiv programmering

Ansats för att identifiera och hantera fel tidigt.

Assert

Kontroll av logiska antaganden i koden.

unittest

Modul för att automatisera och systematisera kodtestning.

Fråga 1: Varför använda assert?

Svar: För att verifiera att programmets tillstånd stämmer och fånga logiska fel tidigt.

Fråga 2: Vad gör unittest-modulen?

Svar: Den tillhandahåller ett ramverk för att organisera och köra automatiserade tester systematiskt.

12.1 Test kap. 1-4

Kom ihåg!!!

- Stil: En förstående stil. Engelska, förklarande namn (*mean_vel* vs *v* vs *mean_velocity*), syntax/semantik
- Reserverade ord: and, if, for, True, False, None
- Identifierare: Börja med bokstav eller understreck, inte siffra
- Hårdkodning: Undvik litteraler spridda i koden, definiera en gång i en tydligt namngiven variabel

Teorifrågor:

1. Vad är en funktions kropp, huvud, parametrar, argument, dokumentsträng
2. Vad är skillnaden på imperativ, funktionell, OOP programmering
3. När används pass vs continue: pass används som en platshållare för framtida kod, medan continue används för att hoppa över resten av koden i en loop och fortsätta med nästa iteration.

4. När används for vs while loopar: for används när man vet hur många iterationer
5. När används .copy() tex för variabler och listor: .copy() används för att skapa en ny kopia av en lista eller ett objekt, vilket gör att ändringar i den nya kopian inte påverkar originalet.
6. När ska man använda en lista vs en dict: array vs hash-tabell om man vill nå top och botten eller allt
7. Vad är hårddisk och operativsystem: hårddisken är en lagringsenhet som används för att permanent lagra data, medan operativsystemet är den programvara tex Windows, Linux, macOS
8. Varför ska man f.close() filer: För att frigöra resurser och säkerställa att alla data skrivs till filen korrekt.
9. När används filerna sys.stdin out derr: tangetnbord, konsol, felmeddelanden
10. Saker jag hoppat över: bra praxis för att programera tillsammans 4.4.3 samt representation av data 4.5

Live Coding:

1. Skapa en python fil. Kör filen samt förbättra denna kod:

```
1      # Denna kod är till för att söka igenom en lista.
2      # Den kollar varje element i sekvensen ett i taget för att
3      # hitta ett specifikt värde.
4      # Om elementet finns i listan returneras positionen där det
5      # hittades.
6
7      target = 10
8
9      def search_element(items, target):
10         global target
11         """
12         Kodan går igenom hela listan och jämför elementen.
13         Om rätt värde hittas sparas indexet.
14         """
15         found_index = -1
16         for i in range(len(items)):
17             if items[i] == target:
18                 found_index = i
19                 pass
20                 break
21             else:
22                 pass
23                 pass
24             return found_index
25
26     # Anrop
27     resultat = search_element([3, 8, 10, 15])
```

2. Hitta alla fel i denna kod:

```
1      räknare = 0
2
3      def skapa_skylt_rapport():
4          ägare = "Karolina"
5          hälsning = "Skylt skapad av" + "Karolina"
```

```

6         dekorationskant = 2 * "="
7         area = 0^2 * 3.14
8
9         if 0.1 + 0.2 == 0.3:
10             print("Precisionen är exakt!")
11
12         if bool(0) = True:
13             print("Status: Giltig radie")
14
15         if area => 50:
16             status_kod = 10
17             if status_kod = 10:
18                 rapport_text = hälsning + " - Stor skylt"
19
20         print(rapport_text)
21         räknare = räknare + 1
22
23     skapa_skylt_rapport()

```

3. Förklara denna kod:

```

1     x, y = 2, 3
2
3     k = x // y
4     r = x % y
5
6     text = f"Resultat:\nKvot: {k}\nRest: {r}"
7     f = float(k)
8     c = 2 + 3j
9     l = x < y
10    t = None
11
12    print(text)
13    print(type(text), type(x), type(f), type(c), type(l), type(
14        t))
15
16    help(int)
17    help(None)

```

4. Brev till framtida jag Skapa ett Python-program som i en loop samlar in användardata (namn, ålder, intressen) och lagrar den i en dictionary för att generera ett "framtidsbrev". Programmet beräknar åldern om 5 år, avgör om nuvarande ålder är jämn eller udda, samt hanterar negativa åldrar med felmeddelanden. Om åldern överstiger 100 år avbryts loopen (break) och ett meddelande om att användaren är en "Du är en legendarisk drake och vet redan allt" skrivs ut.

Slutligen skrivs en sammanfattning ("framtidsbrevet") ut i konsolen baserat på dictionary-datan. Loopen körs tills användaren anger "exit" eller avbryter med Ctrl + C (Keyboard-Interrupt).
5. I denna uppgift ska du arbeta med listan 'data' som skapats i förberedelsekoden. Utför följande operationer i ordning:
 1. Ta fram och skriv ut det näst sista värdet.
 2. Lägg till ett nytt värde i slutet av listan.
 3. Ta bort det 3:e elementet (index 2) med 'del'.

4. Ta bort det 3:e elementet (index 2) med 'pop()' och skriv ut det returnerade värdet.
5. Ta bort alla förekomster av "joker" från listan med 'remove()'.
6. Byt ut det 3:e elementet (index 2) mot ett annat valfritt värde.
7. Skriv ut elementen från index 3 till 9, men bara varannat element.
8. Skriv ut listan baklänges med slicing (steglängd -1).
9. Skriv ut alla element i listan med en 'for'-loop.
10. Skriv ut alla element i listan med en 'while'-loop.
11. Skriv ut hur lång listan är (antal element).
12. Kontrollera om talet 7 finns i listan och skriv ut ett booleskt resultat.
13. Ta fram och skriv ut vilket index "katt" har.
14. Räkna och skriv ut hur många förekomster av "katt" som finns i listan.
15. Vänd på listan baklänges direkt på plats med metoden '.reverse()' och skriv ut den.
16. Lägg till strängen "hund" längst bak i listan med metoden '.insert()'.

Här är koden du behöver:

```

1      # Förberedelsekod: Skapar startlistan med tillräckligt med
      element
2      data = [
3          "apa",
4          "björn",
5          "joker",
6          "katt",
7          7,
8          "joker",
9          "elefant",
10         "katt",
11         "fisk",
12         "gepardt",
13         "häst",
14         "joker",
15         10,
16     ]
17
18     print("Ursprunglig lista:", data)
19     print("-" * 50)

```

6. Skapa en funktion `generera_x(start, stop, step=1)` som returnerar en lista med x-värden med `list(range(start, stop, step))`. Parametern 'step' ska ha standardvärdet 1. Beräkna tillhörande y-värden (t.ex. $y = x^2$) och rita grafen med `matplotlib.pyplot`.
7. Givet meningen: "jag grät när jag inte fick mina björnbär" och de förbjudna bokstäverna: 'ääö'. Skriv ett Python-program som:
 1. Kontrollerar om meningen innehåller någon av de förbjudna bokstäverna.
 2. Identifierar och skriver ut alla enskilda ord i meningen som innehåller någon av dessa bokstäver.
8. Skapa och manipulera en dictionary (uppslagstabell) i Python genom att utföra följande steg: 1. Skapa en dictionary som representerar en frugtlager-status med nycklar (fruktnamn) och värden (antal i lager). 2. Hamta och skriv ut antalet "applen" med hjälp av metoden `.get()`. 3. Uppdatera lagersaldot för "banan" till ett nytt värde. 4. Iterera

igenom hela din dictionary med metoden `.items()` och skriv ut varje nyckel och varde på ett snyggt sätt.

9. I denna uppgift ska du använda `'with open(...)'` som fil för att hantera filer utan att behöva stänga dem manuellt. Börja med att läsa in en textfil med ett kärleksbrev till katter genom att använda metoden `file.readlines()`. Därefter ska du skriva om koden så att alla förekomster av ordet katt ersätts med hund, och spara det nya brevet i en helt ny fil. Avslutningsvis ska du läsa in den nya filen och skriva ut hela dess innehåll i konsolen.

Här är förberedelsekod du behöver:

```
1      # Förberedelsekod: Skapar ursprungsposten
2      meddelande = """Kära katt, du är den underbaraste katt som
3              finns.
4      När en katt spinner blir man alltid glad.
5      Tack för att du är min katt!"""
6
6      with open("karleksbrev_katt.txt", "w") as fil:
7          fil.write(meddelande)
```

10. Skapa en 3x3-matris `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` med nästlade for-loopar. Skriv ut matrisen rad för rad med en nästlad loop, samt beräkna och skriv ut summan av alla element.
11. Gör en while loop som printar nummer och räknar ner från 5 till 0
12. Skriv en funktion som printar odd eller even vad en integer är som tas som input
13. Skriv en funktion som ger boolska värden om ett ord är längre än 8 tecken eller inte

Facit för live coding:

1. Problem 1

```
1      """Algoritm: Lineär sökning""" # FIX: Bytte från # till
2      docstring, lade till algoritmnamn och tog bort för-
3      rklaring av koden
4
3      # FIX: Tagit bort den globala variabeln target = 10 (gjorts
4      om till lokal/parameter)
5
5      def search_element(items, target=10): # FIX: Använder
6      standardvärde target=10 i parameteren
6      # FIX: Tagit bort 'global target' och hanterar target
7      lokalt via parameteren
7      found_index = -1
8      for i in range(len(items)):
9          if items[i] == target:
10             found_index = i
11             # FIX: Tog bort onödig 'pass'
12             break
13             # FIX: Tog bort onödig 'else:' och 'pass'
14             # FIX: Tog bort onödig 'pass'
15             return found_index
16
17      # Anrop
18      resultat = search_element([3, 8, 10, 15]) # Använder
19      standardvärdet 10 automatiskt
```

2. Problem 2

```

1      räknare = 0 # Global variabel
2
3      def skapa_skylt_rapport():
4          global räknare # FIX: Lade till 'global' för att tillå
           ta ändring av den globala variabeln inuti funktionen
5
6          ägare = "Karolina" # FIX: Ändrade enstaka fnutt (')
           till dubbelfnutt (") för att matcha strängens start
7          hälsning = "Skylt skapad av " + ägare # FIX: Lade till
           mellanrum i strängen samt använde variabeln 'ägare'
8          dekorationskant = "=" * 2 # FIX: Ändrade till att
           multiplicera tecknet med antalet gånger (renare stil
           /semantik)
9
10         radie = 3 # FIX: Bytte ut XOR-operatorn (^) mot
           potensoratorn (**) nedan
11         area = radie**2 * 3.14 # FIX: Använder variabeln radie
           istället för hårdkodad 0
12
13         if abs((0.1 + 0.2) - 0.3) < 1e-9: # FIX: Bytte exakt
           likhet (==) mot en toleransjämförelse p.g.a.
           flyttals- / maskinprecision
14             print("Precisionen är exakt!")
15
16         if not bool(0): # FIX: Ändrade = till == (eller 'not
           bool(0)' eller != True) då bool(0) utvärderas till
           False
17             print("Status: Giltig radie")
18
19         rapport_text = hälsning + " - Standard skylt" # FIX:
           Deklarerar variabeln utanför if-blocket så den inte
           får för kort räckvidd (ScopeError)
20
21         if area >= 50: # FIX: Ändrade den felaktiga operatorn
           => till den korrekta >= (större än eller lika med)
22             status_kod = 10
23             if status_kod == 10: # FIX: Bytte
           tilldelningsoperatorn (=) mot jämfö
           relseoperatorn (==) inuti villkoret
24                 rapport_text = hälsning + " - Stor skylt"
25
26         print(rapport_text)
27         räknare = räknare + 1
28
29         '''
30         I detta fall kommer output bli:
31         Skylt skapad av Karolina - Standard skylt
32
33         Om area >= 50, output blir:
34         Skylt skapad av Karolina - Stor skylt
35         '''
36
37     skapa_skylt_rapport()

```

3. Problem 3

```

1      # 1. Parallell tilldelning av int-värden

```

```

2     x, y = 2, 3
3
4     # 2. Heltalsdivision (//) och Modulo/Rest (%)
5     kvot = x // y # Resultat: 0
6     rest = x % y  # Resultat: 2 ty 2 = 0 * 3 + 2
7
8     # 3. Olika datatyper i Python
9     text = f"Resultat:\nKvot: {kvot}\nRest: {rest}" # str med
10    radbrytning (\n)
11    flyttal = float(kvot) # float
12    (0.0)
13    komplext_tal = 2 + 3j # complex
14    är_mindre = x < y # bool (
15    True)
16    inget_värde = None # NoneType
17    (None)
18
19    # Skriv ut strängen
20    print(text)
21
22    # 4. Undersök typer och dokumentation med help() i
23    terminalen
24    print("\n--- Datatyper ---")
25    print(type(text), type(x), type(flyttal), type(komplext_tal),
26    type(är_mindre), type(inget_värde))
27    # Output: <class 'str'> <class 'int'> <class 'float'> <
28    class 'complex'> <class 'bool'> <class 'NoneType'>
29
30    # Använd help() för att slå upp en typ eller variabel
31    help(int)
32    help(None)

```

4. Problem 4

```

1     while True:
2         name = input("Namn (skriv 'exit' för att avsluta): ")
3         if name.lower() == "exit":
4             break
5
6         age_input = input("Ålder: ")
7         if age_input.lower() == "exit":
8             break
9
10        # Kontrollera om inmatningen är siffror (eller ett negativt
11        tal)
12        if age_input.startswith("-") and age_input[1:].isdigit():
13            age = int(age_input)
14        elif age_input.isdigit():
15            age = int(age_input)
16        else:
17            print("Ogiltig inmatning! Vänligen ange ålder med
18            siffror.")
19            continue
20
21        # Logiska villkor för ålder
22        if age < 0:
23            print(
24                """

```



```

23 -----
24 FELMEDDELANDE:
25 Åldern kan inte vara negativ! Vänligen försök igen.
26 -----
27 """
28     )
29     continue
30 elif age > 100:
31     print(
32         """
33 =====
34 Du är en legendarisk drake och vet redan allt!
35 Programmet avslutas nu.
36 =====
37 """
38     )
39     break
40
41 interests = input("Intressen: ")
42 if interests.lower() == "exit":
43     break
44
45 # Lagra informationen i en dictionary
46 user_data = {
47     "namn": name,
48     "alder": age,
49     "framtida_alder": age + 5,
50     "jamn_eller_udda": "jämnt" if age % 2 == 0 else "udda",
51     "intressen": interests,
52 }
53
54 # Utskrift av framtidsbrevet med en flerradig sträng
55 framtidsbrev = f"""
56 =====
57 FRAMTIDSBREV
58 =====
59 Hej {user_data['namn']}!
60
61 Du är idag {user_data['alder']} år gammal, vilket är ett {
62     user_data['jamn_eller_udda']} tal.
63 Om 5 år kommer du att vara {user_data['framtida_alder']} år
64 gammal.
65
66 Dina registrerade intressen:
67 - {user_data['intressen']}
68 """
69 print(framtidsbrev)

```

5. Problem 5

```

1 # Förberedelsekod
2 data = [
3     "apa",
4     "björn",
5     "joker",
6     "katt",
7     7,

```

```

8         "joker",
9         "elefant",
10        "katt",
11        "fisk",
12        "gepardt",
13        "häst",
14        "joker",
15        10,
16    ]
17
18    # 1. Näst sista värdet
19    print("1. Näst sista värdet:", data[-2])
20
21    # 2. Lägg till ett värde i slutet (append)
22    data.append("orm")
23    print("2. Efter tillägg av 'orm':", data)
24
25    # 3. Ta bort det 3:e värdet (index 2) med del
26    del data[2]
27    print("3. Efter del data[2]:", data)
28
29    # 4. Ta bort det 3:e värdet (index 2) med pop och skriv ut
        returvärdet
30    borttaget_varde = data.pop(2)
31    print(f"4. Borttaget värde med pop(2): {borttaget_varde}")
32    print("    Listan efter pop:", data)
33
34    # 5. Ta bort alla 'joker' med remove
35    while "joker" in data:
36        data.remove("joker")
37    print("5. Efter att alla 'joker' tagits bort:", data)
38
39    # 6. Byt ut det 3:e värdet (index 2) mot ett annat värde
40    data[2] = "TIGER"
41    print("6. Efter byte av 3:e värdet till 'TIGER':", data)
42
43    # 7. Print index 3 till 9 men bara varannan (slicing: [3:10:2])
44    print("7. Index 3 till 9 (varannat element):", data[3:10:2])
45
46    # 8. Printa ut listan baklänges med slicing
47    print("8. Listan baklänges (slicing):", data[::-1])
48
49    # 9. Printa ut listan med en for-loop
50    print("9. Utskrift med for-loop:")
51    for item in data:
52        print(item, end=" ")
53    print()
54
55    # 10. Printa ut listan med en while-loop
56    print("10. Utskrift med while-loop:")
57    i = 0
58    while i < len(data):
59        print(data[i], end=" ")
60        i += 1
61    print()
62
63    # 11. Hur lång är listan?
64    print("11. Längd på listan:", len(data))

```

```

65
66     # 12. Finns 7 i listan?
67     print("12. Finns 7 i listan?:", 7 in data)
68
69     # 13. Vilket index har 'katt'?
70     print("13. Index för 'katt':", data.index("katt"))
71
72     # 14. Hur många 'katt' finns i listan?
73     print("14. Antal 'katt' i listan:", data.count("katt"))
74
75     # 15. Printa listan baklänges med .reverse()
76     data.reverse()
77     print("15. Listan efter .reverse():", data)
78
79     # 16. Lägg till en 'hund' på slutet med insert
80     data.insert(len(data), "hund")
81     print("16. Efter .insert() på slutet med 'hund':", data)

```

6. Problem 6

```

1         import matplotlib.pyplot as plt
2
3         # Funktion med standardvärde step=1
4         def generera_x(start, stop, step=1):
5             return list(range(start, stop, step))
6
7         # 1. Standardläge (steg 1)
8         x_val = generera_x(0, 10)
9         y_val = [x**2 for x in x_val]
10
11        # 2. Om användaren anger eget steg (t.ex. steg 2)
12        x_anpassad = generera_x(0, 10, step=2)
13        y_anpassad = [x**2 for x in x_anpassad]
14
15        # Rita grafen
16        plt.plot(x_val, y_val, label="Standard (steg 1)")
17        plt.plot(x_anpassad, y_anpassad, label="Anpassad (steg 2)")
18
19        plt.title("Graf med range() och Matplotlib")
20        plt.xlabel("x")
21        plt.ylabel("y")
22        plt.legend()
23        plt.grid(True)
24        plt.show()

```

7. Problem 7

```

1         mening = "jag grät när jag inte fick mina björnbär"
2         forbjudna_bokstaver = "ääö"
3
4         # 1. Undersök om meningen innehåller någon av bokstäverna
5         har_forbjuden = False
6         for bokstav in forbjudna_bokstaver:
7             if bokstav in mening:
8                 har_forbjuden = True
9
10        print("Innehåller meningen förbjudna bokstäver?:",
11              har_forbjuden)

```

```

11
12     # 2. Hitta och skriv ut orden
13     ord_lista = mening.split()
14     ord_med_forbjudna = []
15
16     for ordet in ord_lista:
17         # Kolla om ordet innehåller å, ä eller ö
18         if (
19             ("å" in ordet)
20             or ("ä" in ordet)
21             or ("ö" in ordet)
22         ):
23             ord_med_forbjudna.append(ordet)
24
25     print("Ord som innehåller förbjudna bokstäver:",
           ord_med_forbjudna)

```

8. Problem 8

```

1     # 1. Skapa en dictionary med nycklar och värden
2     lager = {"äpple": 15, "banan": 8, "päron": 12, "apelsin":
3             20}
4
5     print("Ursprunglig dictionary:", lager)
6
7     # 2. Ta fram ett värde med .get()
8     antal_applen = lager.get("äpple")
9     print("Antal äpplen i lager:", antal_applen)
10
11    # 3. Uppdatera ett värde
12    lager["banan"] = 25
13    print("Efter uppdatering av banan:", lager)
14
15    # 4. Loopa över alla par med .items()
16    print("\n--- LAGERSTATUS ---")
17    for frukt, antal in lager.items():
18        print(f"Frukt: {frukt:<8} | Antal i lager: {antal}")

```

9. Problem 9

```

1     # 1. Läs in kattbrevet med readlines() och with open
2     with open("karleksbrev_katt.txt", "r") as fil:
3         rader = fil.readlines()
4
5     # 2. Ersätt 'katt' med 'hund' på varje rad och spara i en
6     #    ny fil
7     with open("karleksbrev_hund.txt", "w") as ny_fil:
8         for rad in rader:
9             ny_rad = rad.replace("katt", "hund").replace("Katt",
10                 "Hund")
11             ny_fil.write(ny_rad)
12
13    # 3. Läs in den nya filen och printa ut i konsolen
14    with open("karleksbrev_hund.txt", "r") as ny_fil:
15        innehall = ny_fil.read()
16        print(innehall)

```

10. Problem 10

```

1      # 1. Bygg 3x3-matrisen
2      matris = []
3      num = 1
4      for r in range(3):
5          rad = []
6          for k in range(3):
7              rad.append(num)
8              num += 1
9          matris.append(rad)
10
11     # 2. Skriv ut matrisen och beräkna summan
12     total = 0
13     for rad in matris:
14         for tal in rad:
15             print(tal, end=" ")
16             total += tal
17         print()
18
19     print("Summa:", total)

```

11. Problem 11

```

1      # 1. While loop som räknar ner från 5 till 0
2      nummer = 5
3      while nummer >= 0:
4          print(nummer)
5          nummer -= 1

```

12. Problem 12

```

1      def odd_or_even(num):
2          if num % 2 == 0:
3              print(f"{num} är even")
4          else:
5              print(f"{num} är odd")
6
7      # Exempelanrop
8      odd_or_even(4)    # Output: 4 är even
9      odd_or_even(7)    # Output: 7 är odd

```

13. Problem 13

```

1      def is_longer_than_8(word):
2          return len(word) > 8
3
4      # Exempelanrop
5      print(is_longer_than_8("programmering"))    # Output: True
6      print(is_longer_than_8("kort"))             # Output: False

```